

Demo - Random Forest Using Python

In this demo, you will learn how to create Random Forest in Python using scikit-learn library.

Problem Statement

The dataset contains information about 3 brands namely US, Japan, Europe. Other parameters are mpg, cylinders, cubicinches, hp, weightlbs, time-to-60, year. The task is to find the brand of a car using these parameters.

Importing Required Libraries

```
In [ ]: import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import plot_tree
from sklearn.metrics import accuracy_score
```

Dataset

```
In [ ]: #Reading the dataset using pandas
car_data=pd.read_csv('cars.csv')
car_data.head()
```

Data Analysis

```
In [ ]: #Dataset shape to get row and column count
car_data.shape
```

- Checking Null Values

```
In [ ]: #Checking for any null values
car_data.isnull().any()
```

 None of the columns contain null values so there is no need to remove/replace data

- Encoding Target Variable

```
In [ ]: #Using LabelEncoder to encode the target variable
lb=LabelEncoder()
lb.fit(car_data['brand'])
car_data['brand']=lb.transform(car_data['brand'])
```

• Divide Feature And Target Variables

```
In [ ]: #Divide the dataset into independent and dependent variables
X=car_data.drop('brand',axis=1)
y=car_data['brand']
```

• Training And Testing Set

```
In [ ]: #Split the data into training and testing set
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2) #20 percent of data
```

Decision Tree Training

The following parameters should be considered during training a decision tree:

- **criterion:** To measure quality of a split. Supported criteria are "gini" for the Gini impurity and "entropy" for the information gain.
- **splitter:** Either 'best' to choose best split or 'random' to choose random best split. By default takes 'best' value.
- **max_features:** Number of features to consider.
- **max_depth:** The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than min_samples_split samples.

The classifier has several parameter which are used depending on the requirement. For more information about these parameters you can visit: [Decision Tree](#)

```
In [ ]: dt_car=DecisionTreeClassifier(criterion='entropy',splitter='best',max_features=3,max_c
dt_car.fit(X_train,y_train)
```

Visualizing Decision Tree

```
In [ ]: #plot tree can also be used for visualising the tree
plt.figure(figsize=(20,20))
plot_tree(dt_car,class_names=list(lb.classes_),feature_names=list(X_train.columns))
plt.show()
```

```
In [ ]: #Calculating accuracy
pred = dt_car.predict(X_test)
print('Accuracy is:',accuracy_score(y_test,pred))
```

Feature Importance

```
In [ ]: '''
- Selecting only the important features
'''

features=pd.DataFrame({'Features':X_train.columns,'Imp':np.round(dt_car.feature_importances_,2)})
features=features.sort_values('Imp',ascending=False)
```

```
In [ ]: features
```

 The above data shows which feature is important based on criterion provided

 We can select the required important features and retrain the model

Random Forest

The following parameters should be considered before using random forest classifier:

- **n_estimators:** Integer value to define number of trees in the forest.
- **criterion:** To measure quality of a split. Supported criteria are "gini" for the Gini impurity and "entropy" for the information gain.

For information about rest of the parameters: [Random Forest](#)

```
In [ ]: #Training
rf_car=RandomForestClassifier(n_estimators=100, criterion='entropy',max_depth=3)
rf_car.fit(X_train,y_train)
```

```
In [ ]: #Calculating accuracy
pred = rf_car.predict(X_test)
print('Accuracy is:',accuracy_score(y_test,pred))
```

Visualizing

```
In [ ]: #Visualizing a single estimator
plt.figure(figsize=(20,10))
plot_tree(rf_car.estimators_[5],feature_names=X_train.columns,class_names=list(lb.classes_))
plt.show()
```

Conclusion: In this demonstration, we have learned to apply and visualize decision tree algorithm and random forest classifier.